

MENTOR: Guiding Hierarchical Reinforcement Learning With Human Feedback and Dynamic Distance Constraint

Xinglin Zhou¹, Yifu Yuan, Shaofu Yang², *Member, IEEE*, and Jianye Hao³, *Senior Member, IEEE*

Abstract—Hierarchical reinforcement learning (HRL) provides a promising solution for complex tasks with sparse rewards of agents, which uses a hierarchical framework that divides tasks into subgoals and completes them sequentially. However, current methods struggle to find suitable subgoals for ensuring a stable learning process. To address the issue, we propose a general hierarchical reinforcement learning framework incorporating human feedback and dynamic distance constraints, termed MENTOR, which acts as a “mentor”. Specifically, human feedback is incorporated into high-level policy learning to find better subgoals. Furthermore, we propose the Dynamic Distance Constraint (DDC) mechanism dynamically adjusting the space of optional subgoals, such that MENTOR can generate subgoals matching the low-level policy learning process from easy to hard. As a result, the learning efficiency can be improved. As for low-level policy, a dual policy is designed for exploration-exploitation decoupling to stabilize the training process. Extensive experiments demonstrate that MENTOR uses a small amount of human feedback to achieve significant improvement in complex tasks with sparse rewards.

Index Terms—Dynamic distance constraint, hierarchical reinforcement learning, reinforcement learning from human feedback.

I. INTRODUCTION

THE problem of sparse reward is consistently challenging in the domain of reinforcement learning (RL) [1], [2], [3], attributing to two main factors: challenging exploration, and unstable training. In recent years, hierarchical reinforcement learning (HRL) [4], [5], [6], [7] has been proposed as an effective

approach to relieve these issues, especially for long-horizon tasks with sparse rewards, as it decomposes tasks into more manageable subgoals, mitigating challenges related to exploration and unstable training.

There are two primary challenges in its practical applications. 1) *Generating effective subgoals*. To create efficient subgoals that guide low-level policies, manual design [8], [9] and automatic generation methods [6], [10], [11], [12] have been proposed. However, manual design is resource-intensive and struggles with complex tasks [13], while automatic generation demands significant computational resources to explore the entire state space [14]. 2) *Efficient subgoal completion*. During low-level learning, hindsight relabeling [15] adjusts subgoals to turn failed transitions into successes but lacks the capacity for effective exploration. Curiosity-driven techniques, such as Random Network Distillation (RND), prevent revisiting states [16] but can destabilize training due to reward bonuses associated with exploration, particularly in sparse reward settings. Frequent failures in low-level subgoal achievement can lead to non-stationarity at the high level. While some studies [4], [5] address these issues through hindsight mechanisms, e.g., penalizing high-level policies [4], they do not fully resolve the problem of ensuring that low-level policies consistently accomplish tasks.

Extensive efforts have been devoted to identify meaningful subgoals within long-horizon tasks in HRL. This includes studies on options [17], [18], [19], goals [4], [15], [20] and skills [11], [12], [21], [22]. Manual-designed subgoals are costly and challenging for complex tasks [13]. Automatic learning of meaningful subgoals without any guidance from an external expert is a significant challenge [23]. CSD [12] aims to discover latent skills via mutual-information maximization. However, combining meaningful skills into task completion is a continuous challenge. Director [24] introduces a practical method for learning hierarchical behaviors directly from pixels within a learned world model. [25] proposes a skill-based hierarchical reinforcement learning (SHRL) framework for solving the problem of visual navigation of a target. The SHRL framework consists of a high-level policy and three low-level skills: search, adjustment, and exploration. [26] introduces a novel framework called HIGL, which effectively reduces the action space of high-level policies by sampling informative landmarks for exploration and training the high-level policy to generate subgoals towards selected landmarks. HAC [4] addresses the challenges of sparse

Received 11 June 2024; revised 19 October 2024; accepted 26 November 2024. Date of publication 28 January 2025; date of current version 27 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62176056 and Grant 92370132, in part by the Young Elite Scientists Sponsorship Program by the China Association for Science and Technology (CAST) under Grant 2021QNRC001, and in part by Southeast University Big Data Computing Center. (Xinglin Zhou and Yifu Yuan are co-first authors.) (Corresponding author: Shaofu Yang.)

Xinglin Zhou is with the Southeast University-Monash University Joint Graduate School, Southeast University, Suzhou 215123, China (e-mail: 220214855@seu.edu.cn).

Yifu Yuan and Jianye Hao are with the College of Intelligence and Computing, Tianjin University, Tianjin 300072, China (e-mail: yuanyf@tju.edu.cn; jianye.hao@tju.edu.cn).

Shaofu Yang is with the School of Computer Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: sfyang@seu.edu.cn).

Further details and code implementations can be found at <https://github.com/nidesuipao/MENTOR.git>.

Recommended for acceptance by Y. Yu.

Digital Object Identifier 10.1109/TETCI.2025.3529902

reward and non-stationarity at high-levels through the utilization of hindsight action transitions and hindsight goal transitions. HIRO [5] employs a model to generate a new high-level action to rectify the high-level transitions. AGILE [27] addresses the non-stationarity issue by using adversarial learning to guide the generation of compatible subgoals for the low-level policy. Previous work has centered on resolving non-stationary issues or on task decomposition strategies, yet often faces the challenge of sparse rewards, prompting unnecessary exploration. Our study introduces human guidance and DDC to decompose tasks effectively. Human guidance directs subgoals, reducing reward sparsity, while DDC controls subgoal difficulty, synchronizes learning across levels, and relieves non-stationarity issues.

In parallel, the surge in popularity of ChatGPT [28], [29] has significantly boosted the recognition of Reinforcement Learning from Human Feedback (RLHF) in recent times. RLHF is a technique for aligning the functionalities of Large Language Models (LLMs) with human ethical considerations and preference, achieved by integrating human feedback into the learning process [30], [31], [32], [33], [34]. For instance, [35] demonstrates how human preferences can be integrated into text-to-image generation models, aligning them with human aesthetics. [36] introduces a GAN-augmented reinforcement learning strategy that efficiently learns robot behaviors through human preferences, significantly reducing the reliance on human demonstrations. RLHF learns reward functions from pairwise comparison and ranking based on human preference [29], [30], [37], [38]. Human intuition and experience can be incorporated as guidance in high-level decision-making, particularly in setting subgoals within the HRL framework [16]. Nevertheless, humans may struggle to offer immediate guidance that corresponds with the agent's capabilities.

Motivated by aforementioned discussion, in this paper, we introduce a novel framework termed as MENTOR, which integrates human guidance into the high-level policy learning process. This is achieved through RLHF, a method that trains a reward model by learning human preferences via binary comparisons of subgoals. MENTOR utilizes this reward model to generate subgoals at the high-level, effectively steering the agent towards optimal behavior. Additionally, we introduce DDC mechanism, which measures subgoal difficulty by distance and adjusts the subgoal space accordingly. To enable an agent to quickly complete subgoals at the low-level, we introduce Exploration-Exploitation Decoupling (EED), which uses one policy to explore while the other policy learns from the experience of the exploring policy to stabilize the training. We summarize the main contributions as follows:

- We propose MENTOR, leveraging human feedback to guide the subgoal direction and Exploration-Exploitation Decoupling to simultaneously realize exploration and exploitation in subgoal attainment.
- We introduce Dynamic Distance Constraint, dynamic aligning the subgoal difficulty to the capabilities of the low-level policy.
- We demonstrate that MENTOR outperforms other baselines in accomplishing tasks with sparse rewards across various domains.

II. PRELIMINARY

A. Problem Setting

Define Markov Decision Process with a set of goals, characterized by the tuple $\langle \mathcal{S}, \mathcal{G}, \mathcal{A}, P, r, \gamma \rangle$, where $\mathcal{S}$, $\mathcal{G}$, and $\mathcal{A}$ are the sets of state, goal, and action, respectively, $P(s'|s, a)$ is the transition probability function, $r_g(s, g) : \mathcal{S} \times \mathcal{G} \rightarrow \mathbb{R}$ is the reward function, and γ is the discount rate $\in [0, 1)$. The objective is to find an optimal policy π^* such that

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\mathcal{T} \sim \pi} \left[\sum_{t=1}^H \gamma^t r_g(s_t, g_t) \right],$$

where $\mathcal{T}$ denotes a trajectory generated under the policy π starting from an initial state. The reward function can be defined as $r_g(s, g) = \mathbb{1}(\|g - s\|_2 < \epsilon)$, which ϵ is a distance threshold determining contact.

B. Hindsight Relabelling

Hindsight relabelling [15] addresses sparse reward in GCRL by redefining failed transitions as successful transitions, thus generating positive rewards. Specifically, for a failed transition $(s_t, a_t, r_t, s_{t+1}, g)$ with $r_t = r_g(s_{t+1}, g) = 0$, it resets the transition as $(s_t, a_t, 1, s_{t+1}, g')$ with goal $g' = s_{t+1}$. Utilizing this new transition makes it possible to train an off-policy RL algorithm with more positive rewards. [15] also delve into three heuristic methods for goal relabeling to enhance learning efficiency: (1) Future Sampling, which selects goals from subsequent states within the same trajectory; (2) Episode-Specific Random Sampling, which randomly selects goals from the same episode without considering their order; (3) Random Sampling, which chooses new goals from the entire dataset. In our implementation, we have opted for the Future Sampling strategy.

C. Curiosity-Driven Exploration

Intrinsic motivation has been utilized to encourage agents to learn about their surroundings, even when extrinsic rewards are scarce [39], [40], [41]. Curiosity-driven exploration is an intrinsic motivation strategy, exemplified by algorithms like RND, which fosters learning through the agent's desire to discover new information. In this framework, RND [39] serves as the intrinsic reward. RND consists of two neural networks, represented as $f : \mathcal{S} \rightarrow \mathbb{R}^k$ and $\hat{f} : \mathcal{S} \rightarrow \mathbb{R}^k$, which are both randomly initialized and capable of transforming observations into embeddings. By fixing one network and training the other to minimize the prediction error, we follow the distillation optimization process:

$$\mathbb{E}_{s \sim \rho_{\pi(s)}} \min_{\hat{f}} \|\hat{f}(s) - f(s)\|^2, \quad (1)$$

where s is sampled from the replay buffer. Upon the agent's interaction with the environment, yielding the current state s_t , we proceed to compute RND reward:

$$r_{\text{rnd}}(s_t) = \|\hat{f}(s_t) - f(s_t)\|^2. \quad (2)$$

This RND reward encourages the agent to visit novel states, as it will be higher in regions of the state space that the predictor network finds difficult to approximate, thus fostering exploration

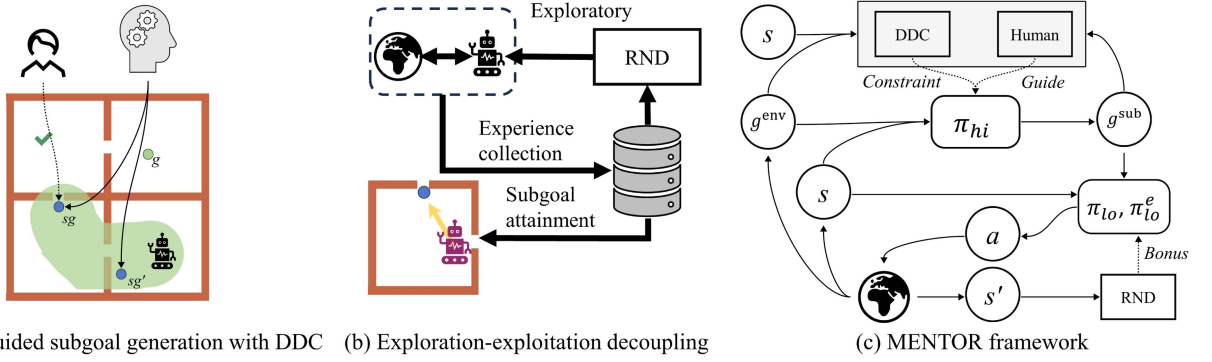


Fig. 1. (a) The high-level policy selects subgoals with DDC (shades of green), and human guides by comparing these subgoals. (b) The low-level decouples exploration and exploitation through two policies, one policy explores the environment and the other learns from the experience of exploration. (c) Diagrammatic representation of MENTOR framework.

in the learning process. To ensure effective policy optimization and stability in reinforcement learning, it is essential that the RND reward, as an intrinsic reward, is normalized to align with the scale of extrinsic rewards:

$$r_{\text{intri}}(s_t) = \frac{r_{\text{rnd}}(s_t) - \mu(r_{\text{rnd}})}{\sigma(r_{\text{rnd}})}. \quad (3)$$

We determine the normalized intrinsic reward for state s_t by converting the RND reward into a Z-score, aligning it with the mean $\mu(r_{\text{rnd}})$ and scaling it according to the standard deviation $\sigma(r_{\text{rnd}})$.

III. METHODOLOGY

HRL consists of multiple levels, each focused on solving specific problems. The high-level is responsible for providing overall guidance, while the low-level handles task execution. This difference in responsibilities between the levels requires distinct reward function characteristics. Depending on unique features of diverse levels, we propose MENTOR, shown in Fig. 1(c). At the high-level, it utilizes RLHF and DDC to address the challenge of generating instructive subgoals shown in Fig. 1(a). At the low-level, it decouples exploration and exploitation, solving the instability associated with curiosity-driven exploration shown in Fig. 1(b).

Before introducing the framework, we briefly review here to introduce notation. The framework consists of four neural networks: high-level policy $\pi_{\text{hi}} : \mathcal{S} \times \mathcal{G} \rightarrow \mathcal{G}$, low-level policy $\pi_{\text{lo}} : \mathcal{S} \times \mathcal{G} \rightarrow \mathcal{A}$, reward model learned by human feedback $r_{\text{hf}} : \mathcal{S} \times \mathcal{G} \times \mathcal{G} \rightarrow [0, 1]$, RND model $f_{\text{rnd}} : \mathcal{S} \rightarrow [0, 1]$ and distance model $d : \mathcal{G} \times \mathcal{G} \rightarrow [0, 1]$ with parameters θ_{hi} , θ_{lo} , θ_{hf} , θ_{rnd} and θ_d respectively. The distance model, denoted as d , operates within the Dynamic Distance Constraint. In a given episode, π_{hi} initially proposes subgoal g_0^{sub} based on current state s and environment goal g^{env} , followed by task given by π_{lo} to achieve g_0^{sub} . If π_{lo} succeeds and the episode remains active, π_{hi} subsequently issues a new subgoal g_1^{sub} . Thus, the high-level trajectory is represented as $(s_{t_0}, s_{t_1}, \dots, s_{t_{n-1}})$, where t_i signifies the moment of the i -th subgoal generation by π_{hi} . Concurrently, the low-level trajectory unfolds as $(s_0, s_1, \dots, s_{L-1})$, L denotes the trajectory length.

A. RLHF and Dynamic Distance Constraint in High-Level

1) *Subgoal Generation Using RLHF*: Subgoal generation poses a significant challenge, with manual setup being costly and difficult, while automatic methods demand extensive computational resources to explore the state space. High-level subgoal generation requires macro, abstract, and generalized guidance, closely related to human preferences. RLHF offers human preferences through sample comparisons. Despite the inherent uncertainty and noise in the human feedback acquired through pairwise preference comparisons, this approach for reward model learning effectively guides the high-level policy learning because HRL mitigates the issue of uncertainty and noisy preferences in RLHF, creating a mutual benefit for both HRL and RLHF.

RLHF uses pairwise comparison to train a reward model which can be used as the reward function for the high-level policy. The training process consists of (1) extracting pairs $(s_1, g_1^{\text{sub}}, g^{\text{env}})$ and $(s_2, g_2^{\text{sub}}, g^{\text{env}})$ from the high-level replay buffer. (2) Human annotators provide pairwise feedback v which 0 prefers $(s_1, g_1^{\text{sub}}, g^{\text{env}})$, 1 prefers $(s_2, g_2^{\text{sub}}, g^{\text{env}})$ and 0.5 for same preference. Preference can be assessed by the distance to the environment goal g^{env} . (3) After collecting the feedback dataset into the reward model buffer B_{hf} , we can train the reward model r_{hf} by optimizing a modified Bradley-Terry objective [42]. We define the possibility that human prefers $(s_1, g_1^{\text{sub}}, g^{\text{env}})$ to $(s_2, g_2^{\text{sub}}, g^{\text{env}})$:

$$P[(s_1, g_1^{\text{sub}}) \succ (s_2, g_2^{\text{sub}}) | g^{\text{env}}] = \frac{\exp(r_{\text{hf}}(s_1, g_1^{\text{sub}}, g^{\text{env}}))}{\sum_{i=1}^2 \exp(r_{\text{hf}}(s_i, g_i^{\text{sub}}, g^{\text{env}}))}. \quad (4)$$

Then, we define two probabilities:

$$p_1 = P[(s_1, g_1^{\text{sub}}) \succ (s_2, g_2^{\text{sub}}) | g^{\text{env}}],$$

$$p_2 = P[(s_2, g_2^{\text{sub}}) \succ (s_1, g_1^{\text{sub}}) | g^{\text{env}}],$$

and the modified Bradley-Terry objective is as follows:

$$\max_{r_{\text{hf}}} \mathbb{E}_{(s_1, g_1^{\text{sub}}, s_2, g_2^{\text{sub}}, g^{\text{env}}, v) \sim B_{\text{hf}}} [(1-v) \log(p_1) + v \log(p_2)]. \quad (5)$$

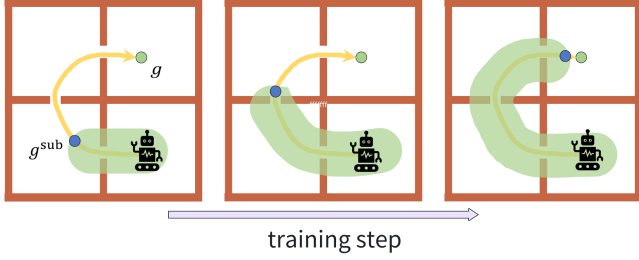


Fig. 2. As the low-level capability improves, the DDC progressively relaxes, allowing the high-level to propose increasingly challenging subgoals.

In optimizing the high-level policy, the high-level reward function is set to be $r_{hi} = r_{hf}(s, g^{sub}, g^{env})$. In the dynamic execution process, humans and algorithms engage in ongoing interactions, where high-level policies can be subject to real-time guidance from human operators.

2) *Dynamic Distance Constraint for Subgoal Difficulty Adjustment*: while humans can provide direction for high-level goals by expressing preferences, they may struggle to define subgoals that align with the low-level policy’s capabilities. It is quite possible that influenced by their cognition, humans might prefer subgoals close to the ultimate goal. Consequently, if the reward model learned by human preferences is designed to assign the highest rewards when the subgoal coincides with the goal, it could lead to a scenario where the high-level policy, without considering the low-level’s capacity and basing its decisions solely on human feedback, generates subgoals that rapidly converge towards the goal. This approach could render the subgoals excessively challenging and diminish their instructional value. Thus multi-level simultaneous learning may be uncoordinated.

To solve this issue above, we introduce DDC, whose function is illustrated in Fig. 2. This method limits the range of subgoals based on their distance from the current state (the variation in green shading depicted in the figure). Under the function of DDC, the learning process of our framework MENTOR is similar to curriculum learning [43], [44], [45]. Curriculum learning facilitates a sequential learning process where simpler subgoals are mastered before progressing to more complex ones, thus laying a solid foundation for advanced subgoal acquisition. This is achieved through a specific formulation:

$$\max_{\pi_{hi}} \mathbb{E}_{g^{sub} \sim \pi_{hi}} \left[\sum_{i=0}^n r_{hi}(s_{t_i}, g_i^{sub}, g^{env}) \right] \\ \text{s.t. } \mathcal{H}(g_i^{sub}, g_{t_i}^{ach}, k) := \max\{d(g_i^{sub}, g_{t_i}^{ach}) - k, 0\} \leq 0, \quad (6)$$

where $d(g_i^{sub}, g_{t_i}^{ach}) : \mathcal{G} \times \mathcal{G} \rightarrow [0, 1]$ represents the distance between the subgoal g_i^{sub} and the achieved goal $g_{t_i}^{ach}$. The parameter k sets the subgoal space range. The constraint ensures that the distance between the high-level subgoal and the current achieved goal remains within a range of k lengths, and we can adjust k to progressively reduce the difficulty. However, in scenarios like the Four Rooms domain, the Euclidean distance as $d(\cdot)$ may not accurately assess the difficulty of completing a task. For instance, a goal in the top right might be hard to reach despite

a low Euclidean distance, indicating the need for a learned step distance measure. DDL [46] offers a methodology for training distance models by randomly sampling state pairs from trajectories, to approximate the number of steps between states. Nevertheless, it’s crucial to evaluate the step distance between unreachable subgoals and the current state. If the low-level policy fails to reach a subgoal, assigning a high distance value to these unreachable subgoals relative to the current state is essential. Neglecting to do so, high-level policies might incorrectly view challenging subgoals as easy, leading to unrealistic subgoal proposals. To tackle this issue, we recommend incorporating extra samples containing unreachable subgoals into the distance model objective:

$$\min_d \mathbb{E}_{\tau \sim \rho_{\pi_{lo}}, i \in [0, L], j \in [i, L], g^{sub} \sim \pi_{hi}(\cdot | s_0, g^{env})} \left[\left(d(s_i, s_j) - \frac{|i - j|}{L} \right)^2 + \mathbb{1}(s_L \neq g^{sub}) (d(s_i, g^{sub}) - 1)^2 \right]. \quad (7)$$

The objective is formulated by minimizing the expected loss across trajectories τ , sampled using the low-level policy $\rho_{\pi_{lo}}$ from recent episodes, and the goals g^{env} drawn from the environment’s distribution $\rho_{g^{env}}$. L is the length of the trajectory.

In (6), optimizing is challenging because of the strict constraint. To overcome this difficulty, we utilize the penalty function method [47], [48], which allows us to establish an unconstrained optimization objective:

$$\max_{\pi_{hi}} \mathbb{E}_{g^{sub} \sim \pi_{hi}} \sum_{i=0}^n [r_{hi}(s_{t_i}, g_i^{sub}, g^{env}) - \alpha \mathcal{H}(g_i^{sub}, g_{t_i}^{ach}, k)], \quad (8)$$

where α is a balancing coefficient to adjust the influence of reward from human guidance and distance constraint. $\mathcal{H}(g_i^{sub}, g_{t_i}^{ach}, k)$ serves as a penalty function that imposes a cost when the subgoal g_i^{sub} deviates from the achievable range defined by the parameter k . This parameter k acts as a threshold, defining the maximum allowable distance for a subgoal to be attainable. This mechanism ensures that the subgoals chosen by the high-level policy are both in harmony with human guidance and within the operational capacity of the low-level policy.

As the capabilities of the low-level improve, it becomes necessary to dynamically adjust the parameter k to ensure that the difficulty of the subgoals remains appropriately challenging. The adjustments in k lead to different optimization goals for the model, which can cause several issues: (1) The coefficient needs to strike a balance between being large enough to ensure constraints are met and small enough to avoid excessive punishment. If the penalty coefficient is set too high, it may overly restrict the solution search space, causing the algorithm to miss the optimal solution or fail to converge. Conversely, if the penalty coefficient is too low, it may not effectively enforce the constraints, leading to solutions that do not satisfy the actual problem constraints. Therefore, a static coefficient may not preserve the optimal balance between human guidance and constraints when k varies. (2) Different values of k lead to distinct distributions of subgoals. Sampling data pairs solely from the comprehensive offline dataset do not ensure that the

reward model accurately assigns rewards to subgoals under the current k . (3) Assuming policy convergence based on a specific k , it's challenging to quickly re-optimize the high-level policy when switching to a new k . In response to these challenges, we have implemented three designs to ensure stability in policy updates when there are changes in k .

Automatic balancing coefficient: We implement a dynamic balancing coefficient which will be updated in real-time to maintain a balance between the rewards and distance constraint. To effectively incorporate the adjustment into MENTOR, we need to optimize two parameters simultaneously: high-level policy θ_{hi} and balancing coefficient α , converting our maximization into a dual problem:

$$\min_{\alpha \geq 0} \max_{\pi_{hi}} \mathbb{E}_{g^{\text{sub}} \sim \pi_{hi}} \sum_{i=0}^n [r_{hi}(s_{t_i}, g_i^{\text{sub}}, g^{\text{env}}) - \alpha \mathcal{H}(g_i^{\text{sub}}, g_{t_i}^{\text{ach}}, k)]. \quad (9)$$

The distance constraint function $\mathcal{H}(\cdot)$ guarantees that the subgoals within a distance k . This function treats subgoals that are within the distance k as having zero effects. However, in the case of α , we want it to decrease when this constraint does not work, and increase when the high-level policies make decisions that do not satisfy this constraint. Since $\mathcal{H}(\cdot)$ is always greater than 0 and the update gradient for α is singular, we need to eliminate the $\max(\cdot)$ function from $\mathcal{H}(\cdot)$ to achieve automatic updates for α . We update the policy π_{hi} firstly and then coefficient α following modification for the optimization:

$$\pi_{hi}^* = \arg \max_{\pi_{hi}} \mathbb{E}_{g^{\text{sub}} \sim \pi_{hi}} \sum_i [r_{hi}(s_{t_i}, g_i^{\text{sub}}, g^{\text{env}}) - \alpha \mathcal{H}(g_i^{\text{sub}}, g_{t_i}^{\text{ach}}, k)], \quad (10)$$

$$\alpha^* = \arg \min_{\alpha} \mathbb{E}_{g^{\text{sub}} \sim \pi_{hi}} \sum_i [-\alpha (d(g_i^{\text{sub}}, g_{t_i}^{\text{ach}}) - k)]. \quad (11)$$

Near-policy sampling: The probability distribution of the data varies depending on the sampling policy and the different values of k , even though all the data exist in an offline experience pool. It is inefficient to train the reward model using all off-policy data. Once the low-level policy has become proficient at achieving subgoals, it becomes redundant to use these subgoals for the training of the reward model, which is intended to enhance the current policy. To address this issue, we propose a new method that involves training the reward model with near-policy data, using (s, g^{sub}, g) pairs that are sampled from recent episodes. This approach allows the reward model to focus on the data that is most relevant to the current policy, enabling a more accurate evaluation of the current policy's performance. Moreover, training a reward model that can accurately evaluate the behavior of the current policy requires fewer samples and training iterations, which results in reduced computational consumption.

Near-policy sampling can also be efficiently executed by merely preserving data from the newest episodes or their respective indices within the experience pool, and subsequently, randomly extracting from these recent datasets when gathering the comparison samples for the reward function training.

High-level exploratory mechanism: As depicted in Fig. 2, the high-level policy may encounter a local optimum after proposing a subgoal in the lower-left region (left panel) and the low-level policy has adapted to achieve it, particularly upon adjusting the k parameter. This policy is guided by a reward model trained on data from the replay buffer and human feedback, lacking initial pre-training. With incremental k values, the high-level policy, potentially already converged, may limit its exploration to a less diverse dataset. This restricted exploration could prevent the identification and storage of superior subgoals in the replay buffer, thereby impeding the enhancement of the reward model and leading to policy training stagnation. To counteract this, it is crucial to promote high-level exploration. We have integrated exploration techniques such as RND and Maximum Entropy (MaxEnt), as detailed in [49]. Specifically, we have adopted the MaxEnt approach, defining the high-level reward as $r_{hi} = r_{hf}(s, g^{\text{sub}}, g^{\text{env}}) - \beta \cdot \log \pi_{hi}(g^{\text{sub}} | s, g^{\text{env}})$, where β is a small constant.

B. Exploration-Exploitation Decoupling in Low-Level Policy

Although the high-level can provide easier goals for the low-level through RLHF, these goals also serve as guidance for the low-level to move forward. However, sparse rewards hinder the low-level ability to quickly achieve subgoals. RLHF is well-suited for the high-level in HRL, but it does not apply to the low-level. If the low-level policy incorporated RLHF, the noise and uncertainty in the reward model could hinder the completion of the task. Existing technologies may use hindsight relabeling, but before the low-level policy explores the subgoals, this technology cannot guarantee that the agent learns how to complete the subtasks. Due to the limited exploration ability of the low-level policy, the agent may fall into local optima and repeatedly explore meaningless areas. Introducing RND can mitigate repeated exploration by promoting novel discoveries, though its direct implementation may result in instability. RND's exploration incentives might lead to neglecting the task of subgoal completion.

To address the issue, we propose EED, a dual-policy program, consisting of an exploration policy π_{lo}^e and a base policy π_{lo} , both sharing the same data buffer. During interactions with the environment, we employ the exploration policy π_{lo}^e and store the experiences in the shared replay buffer. For policy updates, both the exploration and base policies undergo a hindsight relabelling process for relabeled transition data. Subsequently, the exploratory policy is refined by optimizing the following objective function:

$$\max_{\pi_{lo}^e} \mathbb{E}_{\tau \sim \pi_{lo}^e} \left[\sum_{i=1}^L r_g(s_t, g^{\text{sub}}) + r_{\text{intri}}(s_t) \right]. \quad (12)$$

This formulation incorporates an additional element, $r_{\text{intri}}(s_t)$, which introduces curiosity into the reward structure, thereby fostering exploration. Conversely, the base policy is updated by optimizing the following objective function:

$$\max_{\pi_{lo}} \mathbb{E}_{\tau \sim \pi_{lo}} \left[\sum_{i=1}^L r_g(s_t, g^{\text{sub}}) \right]. \quad (13)$$

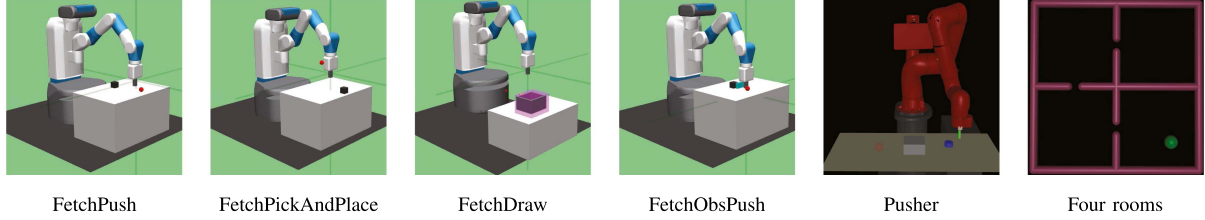


Fig. 3. Experimental Domains for Evaluating MENTOR compared with baselines. The Pusher and Four Rooms serve as representative domains within the class of Partially Observable Markov Decision Processes (POMDPs), characterized by uncertainties in the state observations which require the algorithm to make decisions based on incomplete information. In contrast, the rest domains, such as FetchObsPush, provide a fully observable state space. For further details, please refer to Section A.

This approach enables the base policy to assimilate novel insights from the exploratory data without the inherent burden of the exploration process itself, thus enabling it to focus on the attainment of the defined subgoals.

C. MENTOR Process

Overall, this paper introduces an innovative HRL algorithm, which proficiently identifies an approach for configuring the reward function, RLHF for high-level and EED for low-level. Furthermore, the paper addresses the challenge of inter-layer coordination in HRL by proposing a novel optimization with dynamic distance constraint. In detail, our framework MENTOR works as the following pseudo-code in Algorithm 1. During the interaction with the environment (from line 4 to line 17), a high-level policy π_{hi} is responsible for selecting a subgoal g^{sub} (line 6). Once the subgoal is determined, a low-level exploration policy, denoted as π_{lo}^e , is utilized to execute actions until the subgoal is successfully achieved. This process of selecting subgoals and executing actions continues until the end of the episode. The data obtained from the interaction with the environment, both at the high-level and low-level, is stored in two replay buffers known as B_{hi} and B_{lo} . The model update process is implemented in lines 19 to 23. The high-level policy in Algorithm 3 updates π_{hi} and α alternately. As for the low-level policy, it involves updating the RND model, applying hindsight, updating the low-level base policy π_{lo} , adding the RND bonus in the batch data, and updating the exploration policy π_{lo}^e . From lines 21 to 22, preference tuple pairs are sampled from the data of the last few episodes in B_{hi} , and human labels are obtained and stored in B_{hf} . These batches are then used to update r_{hf} and rewrite the reward data in the high-level buffer B_{hi} . Finally, distance training data is sampled from the data of the last few episodes in B_{hi} and used to update the distance model d . From lines 25 to 34, it tests the low-level base policy success rate on the subgoal g^{sub} given by high-level policy and adjusts the parameters k .

IV. EXPERIMENTS

In this section, we perform a range of experiments in various commonly used domains [50], [51], [52], as depicted in Fig. 3. Through the experiments, we aim to answer the following research questions (RQs):

RQ1 How does the performance of MENTOR in comparison to baseline models in various domains?

RQ2 What is the impact of DDC in MENTOR?

RQ3 What is the impact of human feedback in MENTOR?

RQ4 What insights can be gained about the individual contributions of key components in MENTOR?

A. Setup

Benchmarks: We select FetchPush, FetchPickAndPlace, FetchDraw, FetchObsPush, Pusher, and Four rooms as our simulation benchmarks, widely used in research [15], [16], [52]. As illustrated in Fig. 3, the first five domains involve long-horizon manipulation tasks, while Four rooms focuses on 2D navigation. In our experiments utilizing the RLHF, we employed two distinct approaches to acquire human labels: manual labeling by humans and synthetic labeling through scripts. The specifics of these label acquisition processes are delineated in the Section B. Across the experimental trials, we primarily utilized synthetic labels, with the exceptions explicitly noted.

Hardware: The computer’s hardware is equipped with an AMD Ryzen 7 5700X processor, 32GB of RAM, and an NVIDIA GeForce RTX 3070 Ti graphics card.

Baselines: We have incorporated various learning methods in our baseline implementation, including techniques from RLHF, HRL-HF, distance learning, and hindsight relabelling.

- **RLHF** firstly establishes a reward model based on human feedback and then uses it to optimize policy in reinforcement learning. **PEBBLE** is a RLHF framework that combines unsupervised pre-training and preference-based learning to significantly improve the sample and feedback efficiency of human-in-the-loop reinforcement learning [30]. The key concept of PEBBLE is to learn a reward model on human preference. Unlike the learning strategy of the reward model in the MENTOR, it has an unsupervised pretraining step via intrinsic reward. Here, we apply the state entropy $H(s) = -\mathbb{E} \log p(s)$. By converting the policy to a simpler version, the pretraining reward function is set as $\log(|s_i - s_i^k|)$ in the batch. This implies that maximizing the distance between a state and its nearest neighbor increases the overall state entropy.
- **Hindsight Relabeling** is a data augmentation method for multi-task reinforcement learning that facilitates data sharing across tasks by relabeling experiences to improve sample efficiency. Our baseline, **HER** [15] is a classical hindsight relabeling strategy. HER enables efficient learning from limited and non-diverse rewards by re-framing

Algorithm 1: MENTOR.

```

1: Input: initial state distribution  $\rho(s)$ , goal distribution  $\rho(g)$ ,
   terminal threshold  $\epsilon$ 
2: Initialize high-level policy  $\pi_{hi}$ , low-level base policy  $\pi_{lo}$ ,
   low-level exploration policy  $\pi_{lo}^e$ , reward model  $r_{hf}$ , RND
   model  $f_{rnd}$ , distance model  $d$ , balancing coefficient  $\alpha$ ,
   constraint hyper-parameters  $k$ , high-level buffer  $B_{hi}$ ,
   preference buffer  $B_{hf}$  and low-level buffer  $B_{lo}$ , a high
   threshold high_threshold for the test success rate at the
   value  $k$ , a low threshold low_threshold for the test success
   rate at the value  $k$ .
3: for  $i = 0$  to  $M$  do
4:   receive initial state  $s_t$  and initial  $g_{env}$  from the environment,
      $s_t \leftarrow \rho(s)$ ,  $g_{env} \leftarrow \rho(g_{env})$ 
5:   repeat
6:     select a subgoal  $g^{sub} \leftarrow \pi_{hi}(s_t, g_{env})$ 
7:     store high-level state  $s_{hi} \leftarrow s_t$ 
8:     repeat
9:       select action  $a_t \leftarrow \pi_{lo}^e(s_t, g^{sub})$ 
10:      execute  $a_t$  and observe next state  $s_{t+1}$ 
11:      compute low-level reward  $r_{lo} \leftarrow r_g(s_{t+1}, g^{sub})$ 
12:      store  $(s_t, a_t, s_{t+1}, g^{sub}, r_t)$  to low-level buffer  $B_{lo}$ 
13:       $s_t \leftarrow s_{t+1}$ 
14:    until  $\mathbb{1}(\|g^{sub} - s_t\|_2 < \epsilon)$  or end of episode
15:    calculate the high reward  $r_{hi} \leftarrow r_{hf}(s_{hi}, g^{sub}, g_{env})$ 
16:    append  $(s_{hi}, g^{sub}, s_t, g_{env}, r_{hi})$  to  $B_{hi}$ 
17:     $s_{hi} \leftarrow s_t$ 
18:  until end of episode
19:  TrainHighPolicy( $\pi_{hi}$ ,  $d$ ,  $\alpha$ ,  $B_{hi}$ ) Algorithm 2
20:  TrainLowPolicy( $\pi_{lo}$ ,  $\pi_{lo}^e$ ,  $f_{rnd}$ ,  $B_{lo}$ ) Algorithm 3
21:  near-policy query preference tuples pairs from  $B_{hi}$  and
   store preferences in  $B_{hf}$ 
22:  update  $r_{hf}$  on batches from  $B_{hf}$  using Equation (5) and
   rewrite the reward in  $B_{hi}$ 
23:  near-policy sample distance batches from  $B_{lo}$  and update
   distance model  $d$  using Equation (7)
24:   $success\_rate \leftarrow \{\}$ 
25:  for  $j = 0$  to  $N_1$  do
26:    sample goal  $g$  from environment and select a subgoal  $g^{sub}$ 
27:    execute actions using low-level base policy  $\pi_{lo}$  on subgoal
      $g^{sub}$  and store the success state in  $success\_rate$ 
28:  end for
29:  if  $\text{average}(success\_rate) \geq \text{high\_threshold}$  then
30:    increase  $k$  value
31:  end if
32:  if  $\text{average}(success\_rate) < \text{low\_threshold}$  then
33:    decrease  $k$  value
34:  end if
35: end for

```

Algorithm 2: TrainHighPolicy.

```

1: Input: high-level base policy  $\pi_{hi}$ , distance model  $d$ ,
   balancing coefficient  $\alpha$ , high-level buffer  $B_{hi}$ 
2: for  $i = 0$  to  $N_2$  do
3:   sample a batch  $B$  from buffer  $B_{hi}$ 
4:   use batch  $B$  to update  $\pi_{hi}$  by Equation (10)
5:   use batch  $B$  to update  $\alpha$  by Equation (11)
6: end for

```

Algorithm 3: TrainLowPolicy.

```

1: Input: low-level base policy  $\pi_{lo}$ , low-level exploration
   policy  $\pi_{lo}^e$ , RND model  $f_{rnd}$ , low-level buffer  $B_{lo}$ 
2: for  $i = 0$  to  $N_3$  do
3:   sample a batch  $B$  from buffer  $B_{lo}$ 
4:   use batch  $B$  to update  $f_{rnd}$  by Equation (1)
5:   use  $r_g$  in  $B$  to rewrite the goal and reward
6:   use batch  $B$  to update  $\pi_{lo}$  by Equation (13)
7:   add  $f_{rnd}$  to reward in  $B$  by Equation (3)
8:   use batch  $B$  to update  $\pi_{lo}^e$  by Equation (12)
9: end for

```

failed attempts as successes toward different goals. In our implementation, we utilize a policy to incorporate HER. When sampling batch transitions from the replay buffer, we employ hindsight technology to modify certain parts of the transitions. It is important to mention that the high-level policy differs from the low-level policy of MENTOR in certain aspects. Unlike the low-level policy, the high-level policy solely receives goals from the environment and does not incorporate an RND bonus.

- **HRL-HF** integrates human feedback into the generation of high-level subgoals. We follow the architecture of **HhP** [16]. HhP introduces an HRL method that integrates human preferences and curiosity-driven exploration. By using human preferences to guide decision-making at high-levels and curiosity to promote environmental exploration at low-levels. HhP is considered to be inefficient in combining HRL and RLHF, and it also introduces bias at the low-levels. When we trained HhP, we found that this algorithm was difficult to converge, and to ensure the effectiveness of training, we introduced the hindsight relabelling technique in this algorithm as well.
- **Distance Learning (DL)** learns distance models. The model is utilized to train goal achievement by setting the reward as the negative distance between states and goals. This baseline follows **DDL** [46]. DDL calculates dynamical distances, defining them as the expected number of steps between states in a system. This method leverages unsupervised interactions for learning distances. In the distance evaluation step, the aim is to estimate the dynamic distance between pairs of states visited by a given policy. A distance function, denoted as $d(s_i, s_j)$, is utilized for this purpose. To calculate the distance, multiple trajectories, denoted as τ , are generated by rolling out the policy. Each trajectory has a length of T . The empirical distance between states s_i and s_j in the trajectory τ is then calculated, where $0 \leq i \leq j \leq T$. The empirical distance is simply given by the difference between j and i . To learn the distance function, supervised regression can be employed by minimizing the objective:

$$\mathcal{L}_d = \frac{1}{2} \mathbb{E}_{\tau \sim p_\tau} \left[\sum_{i \in [0, T]} \sum_{j \in [i, T]} (d_\psi(s_i, s_j) - (j - i))^2 \right].$$



Fig. 4. Graphical representation of the success rates for MENTOR in comparison to other baseline methods across different benchmarks on five random seeds. The shaded areas surrounding each curve represent the standard deviation. Within the Four Rooms domain, the performance curve exhibits non-smooth behavior due to the fixed positions of the starting point and the goal. Consequently, the success rate can abruptly transition from 0% to 100%, leading to the curves with large variance. Any curves that are not visible in the graph, indicate a zero success rate throughout the trials. These results are aggregated from an average of five individual runs.

B. Performance Evaluation (RQ1)

In our experiments across six domains, we find that MENTOR excels in learning speed and subgoal attainment as evidenced in Fig. 4. This assessment uses five random seeds and is evaluated over 50 tests. As can be seen from DDL and PEBBLE curves, DDL and PEBBLE rarely learn effectively in complex GCRL environments. DDL’s poor performance may be attributed to the absence of guiding signals and the instability of the reward function. Pairwise comparison guidance makes it difficult for PEBBLE to complete subgoals. HER, a classic GCRL algorithm, serves as a benchmark for evaluating other algorithms. Yet, HhP, despite integrating human guidance and curiosity-driven exploration, underperforms compared to HER in FetchPush. Its benefits are also unclear in Pusher and Four Rooms. This indicates HhP’s inadequate use of human guidance and curiosity in exploration. In contrast, MENTOR, by incorporating Dynamic Distance Constraint and Exploration-Exploitation Decoupling, effectively leverages these elements for utilization of human feedback and exploration, achieving faster and more consistent training than DDL, PEBBLE, and HER. Our analysis highlights the superior performance of MENTOR.

However, MENTOR demonstrates superior performance and is equipped with a greater number of components. To investigate the resource consumption of our algorithm, we conducted additional experiments, meticulously recording the model parameter count and computation time for 1000 iterations of MENTOR and HER in the FetchPush environment. As detailed in Table I, our results reveal that MENTOR has approximately 263% more parameters and requires roughly 176% more running time compared to HER. Although our model has a larger number of components, which significantly increases the parameter count, considering that modern GPUs typically have 8GB or more of memory, these models do not demand excessively high

TABLE I
MODEL PARAMETER COUNT AND TIME CONSUMPTION FOR THE FIRST 1000 EPISODES ON FETCHPUSH DOMAIN

Algorithm	MENTOR	HER
low-level actor&critic	0.560M	0.280M
high-level actor&critic	0.189M	0.189M
reward	0.140M -	-
rnd	0.278M	-
distance	0.068M	-
total count	1.235M	0.469M
time consumption	18.63 ± 0.05min	10.76 ± 0.08min

Here, ‘M’ denotes ‘million’ for the quantity of model parameters, and ‘min’ denotes ‘minutes’ for the duration of time spent.

computational resources. Moreover, by examining the convergence of HER in Fig. 4 and the computational time consumed by both MENTOR and HER algorithms over 1000 episodes, we found that our algorithm consumes 76% more time. However, the convergence speed of our algorithm is more than 3 times that of HER. Therefore, our algorithm remains highly efficient in terms of resource consumption.

C. Impact of DDC (RQ2)

Our study will examine DDC’s effects on learning agents, assessing its impact on agent behaviors and the efficacy of three designs in achieving stable algorithmic convergence. We also explore DDC’s synergism with human feedback.

Examining the correlation between task completion and DDC: As illustrated in Fig. 5, when comparing the black and green curves on the lower side, It is evident that DDC can regulate the difficulty of subgoals provided by limiting the distance. Without DDC, the high-level policy proposes subgoals at random difficulty. By examining this phenomenon in conjunction with the success rate of the subgoals, we can draw the following conclusions: (1) during the initial stages of the training process,

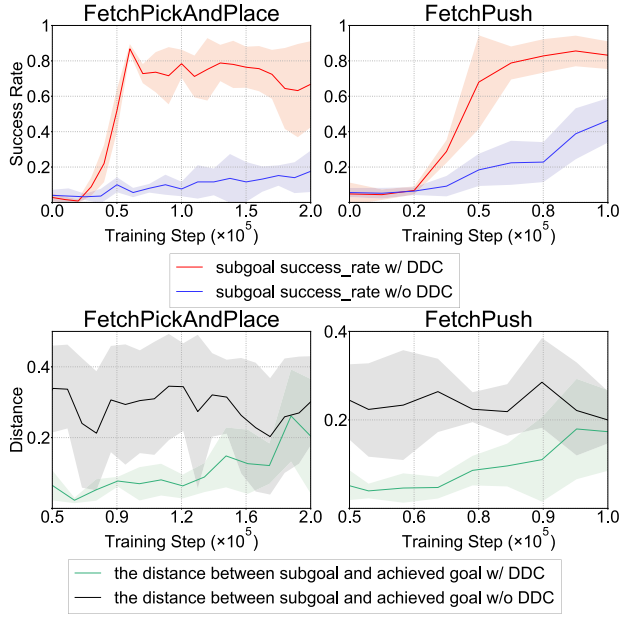


Fig. 5. Impacts of Distance Constraints on success rate in FetchPickAndPlace and FetchPush domains. Since the high-level policy requires data to be collected before updating, a segment is missing from the distance curve.

the low-level policy can rapidly acquire the ability to achieve easy subgoals. (2) Once the low-level policy has successfully mastered a subgoal of a certain difficulty level, it can efficiently progress to learning subgoals of slightly higher difficulty. When there is no DDC, subgoals of randomized difficulty lead to slower learning of the low-level policy. It is concluded that the DDC can efficiently coordinate high-level and low-level.

Analyzing the impact of automatic balancing coefficient: DDC has a mechanism to automatically adjust the balancing coefficient. The mechanism plays a crucial role in balancing the impact of the reward model and distance constraint. As shown in Fig. 6, fixed values of 0.5, 5, and 20 for α are ineffective in learning compared to the automatic adjustment setting. Small α values (0.5 and 5) slow DDC convergence. Large α (20) enhances learning speed but makes it unstable and sensitive to Δk .

To gain more insight into the details of difficulty adjustment under different α , we recorded the change curve of k (Δk of 0.05) under different α settings in Fig. 7. Analyzing the k value change curves for different α values reveals how subgoal difficulty adapts throughout training. These curves track the incremental changes in the k value ($\Delta k = 0.05$) and highlight the influence of the α parameter on the stability and progression of the learning trajectory. Lower α values lead to a more gradual increase in the k value, promoting a stable but slower learning process. In contrast, higher α values cause more pronounced fluctuations, which may accelerate learning but also introduce greater risk of instability. The Auto- α setting demonstrates both rapid and stable k value growth, indicating an adaptive strategy that balances speed and stability in the learning process. These k value curves also serve as indicators of the low-level policy's progress: consistent increases suggest ongoing improvement,

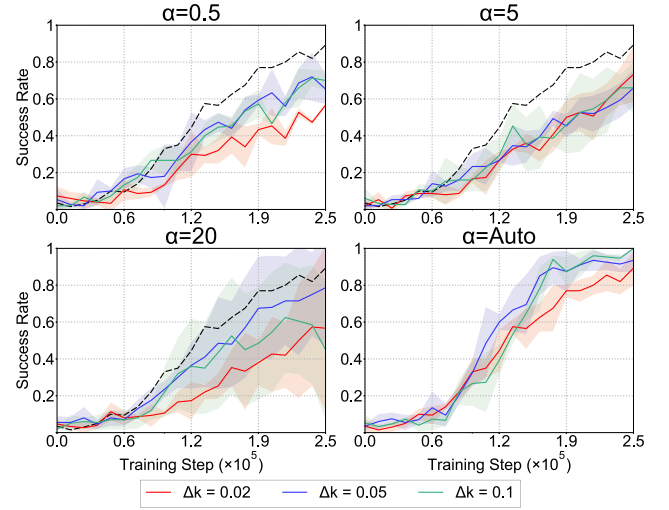


Fig. 6. Effects of the balancing coefficient on the environment goal success rate in FetchPickAndPlace domains are examined on five random seeds. In the first three graphs, the dashed lines represent the average success rate with auto-set α in the worst case, where Δk is 0.02. The adjustment value of k is represented as Δk . We modify the parameter k to increase on successful completion of the subgoal and decrease on failure.

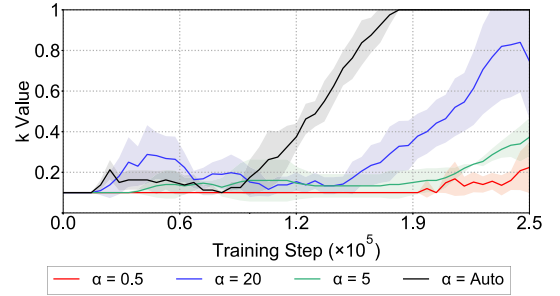


Fig. 7. The variation curve of k value throughout the training process under different α value and Δk is 0.05.

while plateaus or declines may signal the need to reassess the strategy. These analyses highlight the crucial role of the α parameter in shaping the learning process.

Analyzing the near-policy sampling for reward model training and the exploratory mechanism at the high-level: During the training process of MENTOR, we observe some phenomena that the high-level policy cannot be well guided by the rewarding model. These observations can be ascribed to two factors: (1) the reward model's inadequate guidance of the current model, and (2) the increase in the k value leading to the agents in local optima, thereby diminishing further exploration of potential subgoals. In Fig. 8, the reward heatmap, shaped by human feedback, approximates the oracle function. We observe that when near-policy samples are absent, the reward model tends to emphasize the entirety of the state space. This, in turn, makes it more challenging and computing resource-consuming for the reward model to effectively learn.

Additionally, we have observed that there are instances where the high-level policy becomes stuck in local optima as we incrementally increase the value of k for a long time. This

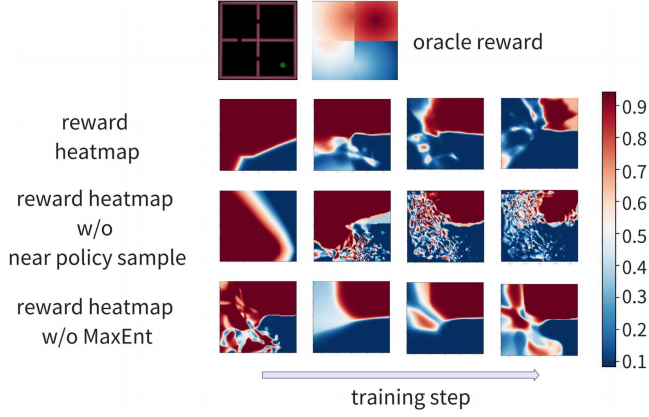


Fig. 8. Near-policy sampling and exploratory mechanism effects on reward model in the Four rooms domain. On the top, the oracle reward function in the Four rooms domain indicates where the agent likely earns higher rewards, guiding it from the lower right to the upper right corner. We use the oracle reward to generate synthetic labels to train the reward model following (5). The heatmaps below the oracle reward reflect how the reward model adjusts in response to the agent’s exploration and policy updates within the environment. The heatmaps of reward model for the top and bottom exhibit changes across 2000 episodes, whereas the heatmap for the center demonstrates alterations throughout 10000 episodes.

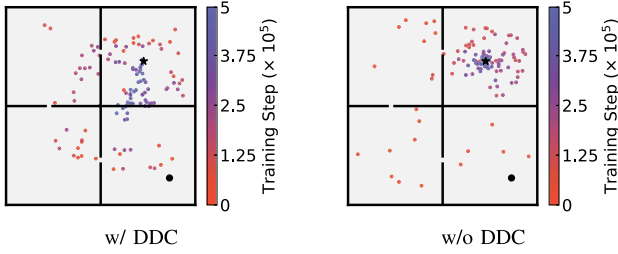


Fig. 9. In Four rooms domain, we compare subgoal distributions with and without DDC during training. Subgoals are shown as colored circles, with a red-to-blue gradient for training time. The starting point is marked by a black circle in the lower right, while the ending point is a pentagram in the upper left.

phenomenon is attributed to two reasons: (1) it lacks pretraining at the high-level, and (2) the high-level lacks the exploratory ability, leading to not trying new subgoals. Pretraining plays a crucial role in RLHF, yet it can be inefficient at the high-level due to limited samples (often only one per episode). In the operation of the high-levels, we find it necessary to increase the exploratory ability. As shown in Fig. 8, reward learning becomes challenging without MaxEnt (maximum entropy). We compare RND and MaxEnt in terms of exploratory at high-levels. After conducting experiments displayed in Fig. 10, we discovered that MaxEnt outperforms RND. While RND promotes the exploration of novel states, MaxEnt focuses on improving action entropy. The combination of the reward model and DDC can lead to previously disregarded decisions being recognized as good decisions. RND can learn and incorporate these decisions previously, treating them as non-novel at late times. However, MaxEnt does not encounter this issue and as a result, it achieves significantly better performance compared to RND.

Analyzing the overlay effects of DDC and human feedback: Having investigated DDC’s impact, we now turn to assess the

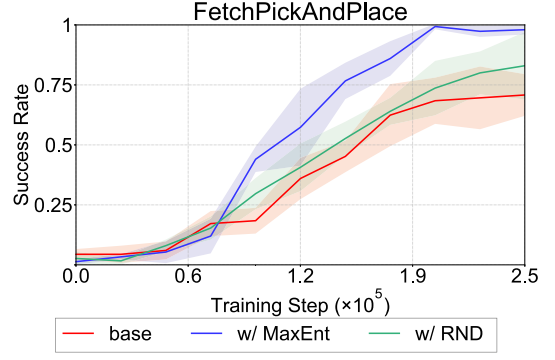


Fig. 10. Exploratory mechanism effects analysis in FetchPickAndPlace domain.

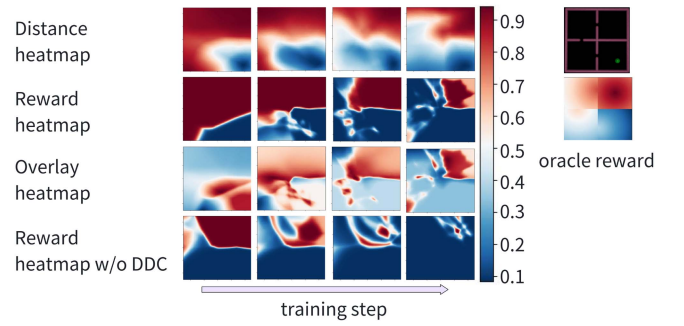


Fig. 11. DDC effects on reward model in the Four rooms domain. The heatmaps serve as visual representations of the models’ learning progress, where the distance model learns to accurately estimate the state to initial state distances and the reward model learns to assign values that incentivize the agent’s progression towards the final goal. The overlay reward heatmap captures the integrated effect of both the reward model and the distance model, as articulated in the adjustments of the high-level policy update detailed in (9).

combined influence of human guidance and DDC on subgoal formulation. Fig. 9 depicts the training phase subgoal distribution, both with and without DDC. To the right of the figure, DDC’s inactivity results in a fragmented learning interaction between levels. While the high-level, aided by human guidance, swiftly navigates to the goal, it neglects the low-level’s execution capabilities, leading to transient and inefficient guidance, evident in the predominance of red subgoals in most regions except the upper-right corner. In contrast, on the figure’s left, with DDC active, the high-level delineates a subgoal sequence starting from the lower right, proceeding to the lower left, then the upper left, and culminating at the upper right corner. This strategy, different from the scenario without DDC, significantly bolsters the efficiency of human guidance, contributing to MENTOR overall effectiveness.

We also analyze the heatmap of the reward and distance model. The analysis of the heatmaps in Fig. 11 shows complex interactions between the distance and reward models in the MENTOR framework. The distance model assesses subgoal difficulty. This overlap, calculated with $r_{hf}(s, g^{sub}, g) + \alpha H(g^{sub}, g, k)$, identifies areas optimizing rewards in alignment with the low-level policy’s capabilities. However, without DDC, the human feedback-based reward model is limited, indicating

TABLE II
CROSS-ANALYSIS ON EPISODES (100 STEPS PER EPISODE) TO THE SUCCESS OF
VARIABLES BATCH QUERIES AND QUERY FREQUENCY

Batch Queries	Query Frequency		
	25	50	100
0	3522±1441	-	-
10	1565±128	1765±268	1820±261
25	1455±165	1770±108	1683±203
50	1406±55	1735±253	1760±277

For each combination of variables, we ran 5 trials on different seeds and recorded the mean and variance.

only high-reward areas without guiding the agent on how to reach them. This can also corroborate the subgoal distribution in Fig. 9.

D. Impact of Human Feedback (RQ3)

This part is dedicated to examining the effects of human feedback on Model MENTOR. Firstly, we evaluate the influence of the frequency and quantity of human feedback on algorithmic performance. Following this, we examine the differences between real human labels and synthetic labels, explain the rationale for using synthetic labels in our experiments, and demonstrate the practical applicability of our algorithm to real-world contexts.

Analyzing the quantity and frequency of feedback: In Table II, the data shows the number of training episodes needed for 100% task success across different query frequencies and batch sizes. The table illustrates that integrating human feedback into training markedly improves learning speed, as highlighted by the contrast between label-free experiments and those with feedback. Further analysis demonstrates a consistent pattern: higher feedback frequency and quantity correlate with increased success rates. When considering the total labels and their impact on learning speed, our algorithm significantly enhances efficiency and stability by requiring only a small number of labels: 10 per 100 episodes, totaling 180, compared to experiments with non-feedback.

Comparing human collected labels and synthetic labels: In our study, we employ two methods for providing human guidance: synthetic labels generated through scripted techniques and human-generated labels.

We monitored the disagreement rates between authentic human labels and synthetic labels, as well as the accuracy of the reward models, as shown in Fig. 12. Given that the starting and ending positions in the Four rooms domain are predetermined, we provided 10 labels every 25 episodes. Conversely, in FetchPush where positions are not fixed, we provided 50 labels every 50 episodes. The data reveals discrepancies between human and synthetic labels, suggesting that human labeling is susceptible to errors due to subjective factors. Furthermore, the RLHF method produced a model whose accuracy converged to approximately 80%, indicating that even the trained model has its uncertainties and potential inaccuracies. This is also supported by the difference between the heatmap of the trained reward model and the heat map of the oracle reward model in Fig. 11. However, our algorithm demonstrates sufficient robustness to handle the inaccuracies inherent in RLHF. This implies that integrating

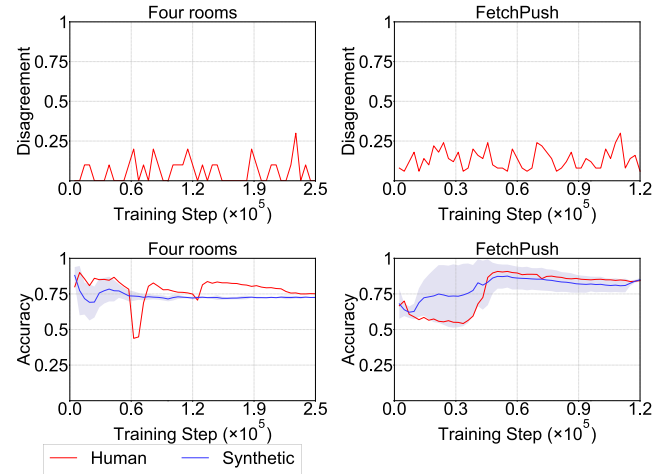


Fig. 12. On the top side, it records the rate of disagreement between human annotations and synthetic labels. There lies a 10% to 20% disparity between human-labeled labels and synthetic labels. On the bottom side, we present the accuracy rate on the training data of the reward model. Within the Four room domain, the reward model's accuracy is 75%, and approximately 80% for FetchPush.

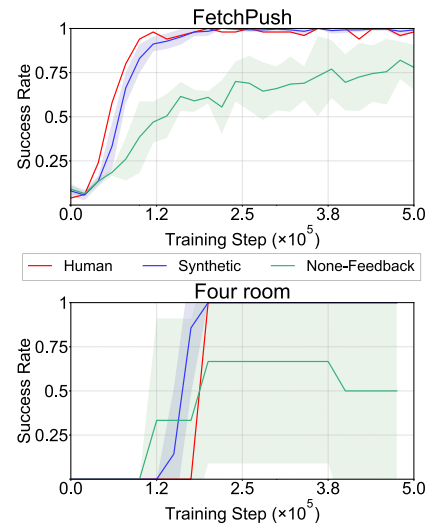


Fig. 13. Experiments for evaluating MENTOR with script-generated and human-collected labels on the FetchPush and Four rooms domain. In these experiments, we provided 10 labels every 25 episodes. The results for the script-generated labels and non-feedback scenarios were based on five random seeds to ensure robustness, while the human-collected label experiment relied on a single random seed.

RLHF into the high-level policy is a highly compatible approach; on the one hand, RLHF fulfills the reward requirements of the high-level, and on the other hand, the low-level policy remains unaffected by the inaccuracies of RLHF. During the algorithm testing, the low-level policy completes tasks independently, hence the final performance of the algorithm is not affected.

We compare the performance of human collected labels and synthetic labels in FetchPush and Four rooms domains. As illustrated in Fig. 13, models trained with human labels exhibited comparable learning rates, with performance differences potentially attributed to noise in human feedback. After comparing

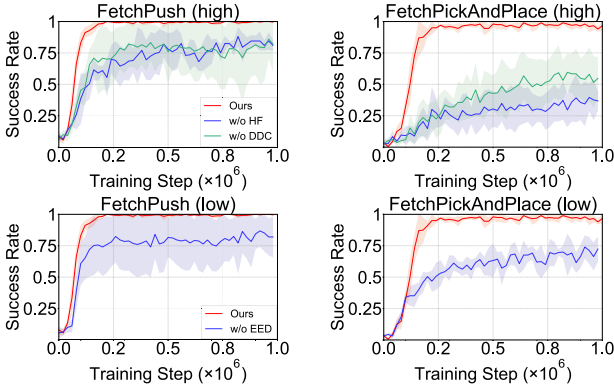


Fig. 14. Ablations studies for MENTOR in FetchPickAndPlace and FetchPush domains at five random seeds. The ablation experiment involved immobilizing the low-level policy, removing the HF and DDC functions of the high-level policy, immobilizing the high-level policy, and removing the EED at the low-level.

the experiments with human collected labels to the experiments with non-feedback, we can find a significant improvement in the algorithm’s convergence speed and final performance. Therefore, we conclude that synthetic labels have a similar effect on improving the algorithm as real human labels. This implies that in other experiments, synthetic labels could potentially serve as a replacement for human collected labels.

E. Ablation Studies (RQ4)

The ablation study in the MENTOR framework, focusing on the FetchPickAndPlace and FetchPush domains, evaluates how components like HF (human feedback), DDC, and EED (Exploration-Exploitation Decoupling) contribute to learning efficiency and goal achievement. Fig. 14 (top) assesses each module’s effectiveness by systematically removing high-level modules and observing the impact on model performance. The removal of DDC and human feedback results in slower convergence, reduced performance, and decreased stability. The study mentioned in Fig. 14 (bottom) discovered that the absence of EED negatively affects the balance between exploration and exploitation, resulting in lower success rates even after algorithm convergence. This emphasizes the significance and interdependence of these modules in improving learning for complex tasks in the MENTOR framework.

V. CONCLUSION

This study presents MENTOR, an innovative method that combines human feedback and Dynamic Distance Constraint for learning guidance. It integrates high-level human insights for selecting subgoals concerning low-level capabilities and introduces exploration-exploitation decoupling at the low-level to improve training stability and efficiency. Our experiments demonstrate the framework’s effectiveness in complex tasks with sparse rewards, outperforming existing baselines.

We recognize the complexity of human guidance beyond just subgoal selection and aim to explore a wider range of feedback

integration to enhance learning dynamics. We plan to further expand the framework’s applications and refine its mechanisms, aiming to advance hierarchical reinforcement learning and create more intuitive, adaptable learning systems.

APPENDIX A DOMAINS DETAILS

In this section, we will provide further elaboration on the benchmarks utilized for comparing our method with the baselines. Specifically, we will delve into the details of the observation space, action space, and the configuration of the reward function.

FetchPush: In this domain, the aim is to use a 7-DoF Fetch Mobile Manipulator, equipped with a closed two-fingered parallel gripper, for transporting a block to a set position on a table. The robot’s movement is finely adjusted using Cartesian coordinates, and the MuJoCo framework calculates the inverse kinematics. This task, which involves pushing the block with the gripper constantly closed, is ongoing, requiring the robot to steadily keep the block at the target position. The scenario is observed through a 25-element array, encompassing kinematic details of both the block and gripper. The action space is a $\text{Box}(-1.0, 1.0, 4, \text{float32})$, with each action altering the gripper’s Cartesian position (dx, dy, dz) and managing its opening and closing. The reward system applies -1 when the block isn’t at the target, and 0 when correctly positioned, defined as being within 0.05 meters of the target.

FetchPickAndPlace: Utilizing the same robot setup with FetchPush, this domain focuses on moving a block to a defined point, including mid-air locations. It shares the same observation array and action space as FetchPush, with the addition of a goal-aware observation space. The reward system remains consistent with the FetchPush domain.

FetchDraw: Utilizing the same 7-DoF Fetch Mobile Manipulator with a two-fingered parallel gripper, this task involves the robot’s precise interaction with a drawer. The objective is two-fold: (1) to reach for the drawer handle, and (2) to slide the drawer to a target position by pulling or pushing the handle. The manipulation requires the gripper to perform open and close actions for a firm grasp on the drawer handle. The robot must successfully move and maintain the drawer at the target position indefinitely. The reward system remains consistent with the FetchPush domain.

FetchObsPush: This task engages the same 7-DoF Fetch Mobile Manipulator, which is equipped with a two-fingered parallel gripper. The robot is tasked with manipulating a block to a desired position on a table in the presence of an obstacle. The challenge requires the robot to (1) approach and securely grasp the block, and (2) navigate and push the block to the target location, accounting for the obstacle whose size and shape are unknown. This task demands precision handling and adaptability to avoid obstacle while ensuring the block reaches its target position. As with the FetchPush, FetchObsPush is continuous, with the robot required to keep the block within 0.05 meters of the target position indefinitely. The reward system remains consistent with the FetchPush domain.

Pusher: This domain involves the manipulation of a robotic arm, specifically a sawyer robotic arm, in an environment with multiple obstacles. The objective is to successfully push an obstacle, referred to as a puck, to a designated goal area marked by a red dot. Unlike the FetchPush problem, which has a 25-dimensional observation, the state space in this environment is determined by the position of the puck and the arm. The action space, on the other hand, involves controlling the position of the robotic arm. It is a discrete 9-dimensional space where each action corresponds to a delta change in the position in a 2-D space. The reward system remains consistent with the FetchPush domain.

Four rooms: In the four-room domain, the agent's task is to navigate to a specified goal while dealing with initially unknown obstacles. The agent is positioned at $(0.4, -0.4)$ in the bottom right room, aiming for the goal at $(0.25, 0.25)$ in the top right room. The state observation is the agent's precise (x, y) location, and it has a set of 9 possible actions to choose from, which allow movement in all cardinal and diagonal directions, or to remain stationary. Key to this task are the doorways in the walls, which are the sole means for the agent to traverse between rooms.

In the FetchPush, FetchPickAndPlace, FetchDraw, FetchObservePush, and Pusher domains, synthetic labels are generated using a dense sparse approach. This means that the reward returned is calculated as the negative Euclidean distance between the achieved goal position and the desired goal. In the Four rooms domain, we utilize the reward function displayed in (14). The reward heatmap, referred to as the oracle reward model, is depicted in Fig. 11. In the top right quadrant, the agent receives a reward based on the negative Euclidean distance from its current position s to the goal $[0.25, 0.25]$. In the top left quadrant, the reward is the negative Euclidean distance from s to a fixed point $[0, 0.3]$, with an additional penalty of -0.3 . In the bottom left quadrant, the reward is the negative Euclidean distance from s to $[-0.3, 0]$, with a penalty of -0.6 . In the bottom right quadrant, it is the negative Euclidean distance from s to $[0, -0.3]$, with a penalty of -1 .

$$r_{\text{Four rooms}}(s) = \begin{cases} -\|s - [0.25, 0.25]\|_2 & \text{if } s_x \geq 0 \text{ and } s_y \geq 0, \\ -\|s - [0, 0.3]\|_2 - 0.3 & \text{if } s_x \leq 0 \text{ and } s_y \geq 0, \\ -\|s - [-0.3, 0]\|_2 - 0.6 & \text{if } s_x \leq 0 \text{ and } s_y \leq 0, \\ -\|s - [0, -0.3]\|_2 - 1 & \text{if } s_x \geq 0 \text{ and } s_y \leq 0. \end{cases} \quad (14)$$

APPENDIX B DETAILS OF IMPLEMENTATION

a) Labels acquisition: We obtain labels through two ways: human annotation and script synthesis. The script-synthesized labels are generated based on dense reward functions.

After extracting tuple pairs from the replay buffer B_{hi} in line 22 of Algorithm 1, such as $(s_{hi1}, g_1^{\text{sub}}, g_1)$ and $(s_{hi2}, g_2^{\text{sub}}, g_2)$, the subsequent step involves obtaining preference labels through interaction with humans or a script. We have designed an interactive interface for human annotation, see Fig. 15. This interface is constructed based on the Four rooms domain, where

TABLE III
HYPERPARAMETERS OF MENTOR

Hyperparameters	Values
High-level policy	
Actor learning rate	3×10^{-4}
Critic learning rate	3×10^{-4}
Replay buffer size	10^6
Hidden layers	3
Hidden size	256
Batch size	256
Soft update rate	0.005
Policy update frequency	1
γ	0.95
Distance model learning rate	3×10^{-4}
Distance model replay buffer size	1000
Distance model hidden layers	3
Distance model hidden size	256
Distance model batch size	256
Reward model learning rate	3×10^{-4}
Reward model replay buffer size	1000
Reward model hidden layers	3
Reward model hidden size	256
Reward model batch size	256
Query Frequency	50
Batch Queries	50
Success rate high threshold	0.6
Success rate low threshold	0.3
Δk	0.05
Initial k	0.05
β	0.1
Low-level policy	
Actor learning rate	10^{-3}
Critic learning rate	10^{-3}
Hidden layers	3
Hidden size	256
Replay buffer size	10^6
Batch size	512
Soft update rate	0.005
Policy update frequency	1
γ	0.95
RND learning rate	3×10^{-4}
RND hidden layers	3
RND hidden size	256
RND represent size	512
RND bonus scaling	1.0
Hindsight sample ratio	0.8

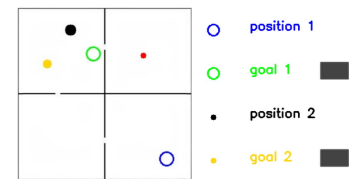


Fig. 15. A simple interface for human annotation.

it differentiates tuple pair information through distinct colors of hollow and solid circles. Since the goal position is fixed within this domain, we represent the goal located in the top right corner with a red solid circle. When an annotator needs to determine which tuple is more advantageous for policy learning, they can either click the gray button situated to the right of the letters “goal” or input ‘0’ or ‘1’ on the keyboard. If the annotator cannot distinguish which tuple is good or bad, he can enter the ‘2’ on the keyboard, and we will set the label to 0.5 to indicate the same preference.

We employ a script to fulfill the role of annotators. The script will calculate the dense reward using an oracle reward function. In the first five domains, we employ the Euclidean distance between the subgoal and the goal. In the Four rooms domain, we use the reward function as mentioned in (14). Depending on the magnitude of the calculated reward for different tuples, we can assign the label to the tuple corresponding to the higher reward.

b) *Hyperparameters*: see Table III.

REFERENCES

- [1] S. Reddy, A. D. Dragan, and S. Levine, "SQL: Imitation learning via reinforcement learning with sparse rewards," in *Proc. 8th Int. Conf. Learn. Representations*, 2020, pp. 10092–10105.
- [2] M. Vecerik et al., "Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards," 2017, *arXiv:1707.08817*.
- [3] Q. Luo and T.-P. Tan, "RARSMSDou: Master the game of DouDiZhu with deep reinforcement learning algorithms," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 1, pp. 427–439, Feb. 2024.
- [4] A. Levy, G. D. Konidaris, R. Platt Jr., and K. Saenko, "Learning multi-level hierarchies with hindsight," in *Proc. Int. Conf. Learn. Representations*, New Orleans, LA, USA, 2019, pp. 4377–4392.
- [5] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Montréal, Canada, 2018, pp. 3307–3317.
- [6] M. Hutsebaut-Buysse, K. Mets, and S. Latré, "Hierarchical reinforcement learning: A survey and open research challenges," *Mach. Learn. Knowl. Extraction*, vol. 4, no. 1, pp. 172–221, 2022.
- [7] T. Ma, K. Peng, H. Rong, Y. Qian, and N. Al-Nabhan, "Hierarchical coordination multi-agent reinforcement learning with spatio-temporal abstraction," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 1, pp. 533–547, Feb. 2024.
- [8] T. G. Dietterich, "Hierarchical reinforcement learning with the MAXQ value function decomposition," *J. Artif. Intell. Res.*, vol. 13, pp. 227–303, 2000.
- [9] R. Parr and S. Russell, "Reinforcement learning with hierarchies of machines," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 1997, pp. 1043–1049.
- [10] T. Zhang, S. Guo, T. Tan, X. Hu, and F. Chen, "Adjacency constraint for efficient hierarchical reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4152–4166, Apr. 2023.
- [11] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine, "Diversity is all you need: Learning skills without a reward function," in *Proc. Int. Conf. Learn. Representations*, New Orleans, LA, USA, 2019, pp. 5227–5248.
- [12] S. Park, K. Lee, Y. Lee, and P. Abbeel, "Controllability-aware unsupervised skill discovery," in *Proc. Int. Conf. Mach. Learn.*, Honolulu, Hawaii, USA, 2023, pp. 27225–27245.
- [13] M. Ahn et al., "Do as i can, not as i say: Grounding language in robotic affordances," 2022, *arXiv:2204.01691*.
- [14] V. Campos, A. Trott, C. Xiong, R. Socher, X. Giró-i Nieto, and J. Torres, "Explore, discover and learn: Unsupervised discovery of state-covering skills," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1317–1327.
- [15] M. Andrychowicz et al., "Hindsight experience replay," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 5048–5058.
- [16] N. Bougie and R. Ichise, "Hierarchical learning from human preferences and curiosity," *Appl. Intell.*, vol. 52, pp. 7459–7479, 2022.
- [17] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 1726–1734.
- [18] J. Harb, P.-L. Bacon, M. Klissarov, and D. Precup, "When waiting is not an option: Learning options with a deliberation cost," in *Proc. AAAI Conf. Artif. Intell.*, 2018, Art. no. 387.
- [19] A. Bagaria and G. Konidaris, "Option discovery using deep skill chaining," in *Proc. Int. Conf. Learn. Representations*, Addis Ababa, Ethiopia, 2020, pp. 7114–7134.
- [20] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2015, pp. 1312–1320.
- [21] A. Sharma, S. Gu, S. Levine, V. Kumar, and K. Hausman, "Dynamics-aware unsupervised discovery of skills," in *Proc. 8th Int. Conf. Learn. Representations*, Addis Ababa, Ethiopia, 2020, pp. 13207–13227.
- [22] A. Levy, S. Rammohan, A. Allievi, S. Niekum, and G. Konidaris, "Hierarchical empowerment: Towards tractable empowerment-based skill learning," 2023, *arXiv:2307.02728*.
- [23] Y. Jinnaï, D. Abel, D. Hershkowitz, M. Littman, and G. Konidaris, "Finding options that minimize planning time," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3120–3129.
- [24] D. Hafner, K.-H. Lee, I. Fischer, and P. Abbeel, "Deep hierarchical planning from pixels," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 26091–26104.
- [25] S. Wang, Z. Wu, X. Hu, Y. Lin, and K. Lv, "Skill-based hierarchical reinforcement learning for target visual navigation," *IEEE Trans. Multimedia*, vol. 25, pp. 8920–8932, 2023.
- [26] J. Kim, Y. Seo, and J. Shin, "Landmark-guided subgoal generation in hierarchical reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 28336–28349.
- [27] V. H. Wang, J. Pajarinen, T. Wang, and J. Kämäräinen, "Hierarchical reinforcement learning with adversarially guided subgoals," 2022, *arXiv:2201.09635*.
- [28] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [29] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.
- [30] K. Lee, L. M. Smith, and P. Abbeel, "PEBBLE: Feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 6152–6163.
- [31] K. Lee, L. M. Smith, A. D. Dragan, and P. Abbeel, "B-PREF: Benchmarking preference-based reinforcement learning," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, Dec. 2021, pp. 1–15.
- [32] J. Park, Y. Seo, J. Shin, H. Lee, P. Abbeel, and K. Lee, "SURF: Semi-supervised reward learning with data augmentation for feedback-efficient preference-based reinforcement learning," in *Proc. 10th Int. Conf. Learn. Representations*, 2022, pp. 1–17.
- [33] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2023, pp. 53728–53741.
- [34] J. Dai et al., "Safe RLHF: Safe reinforcement learning from human feedback," in *Proc. 12th Int. Conf. Learn. Representations*, Vienna, Austria, 2024, pp. 1–28.
- [35] X. Wu, K. Sun, F. Zhu, R. Zhao, and H. Li, "Human preference score: Better aligning text-to-image models with human preference," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 2096–2105.
- [36] H. Zhan, F. Tao, and Y. Cao, "Human-guided robot behavior learning: A GAN-assisted preference-based reinforcement learning approach," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 3545–3552, Apr. 2021.
- [37] N. B. Shah et al., "Estimation from pairwise comparisons: Sharp minimax bounds with topology dependence," *J. Mach. Learn. Res.*, vol. 17, no. 58, pp. 1–47, 2016.
- [38] A. Saha and A. Krishnamurthy, "Efficient and optimal algorithms for contextual dueling bandits under realizability," in *Proc. Int. Conf. Algorithmic Learn. Theory*, 2022, pp. 968–994.
- [39] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Exploration by random network distillation," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, pp. 1–17.
- [40] M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, "Unifying count-based exploration and intrinsic motivation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1479–1487.
- [41] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2778–2787.
- [42] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I the method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.
- [43] Y. Zhang, P. Abbeel, and L. Pinto, "Automatic curriculum learning through value disagreement," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, Art. no. 641.
- [44] P. Klink, C. D'Eramo, J. Peters, and J. Pajarinen, "On the benefit of optimal transport for curriculum reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 11, pp. 7191–7204, 2024.
- [45] G. Tzannetos, P. Kamalaruban, and A. Singla, "Proximal curriculum with task correlations for deep reinforcement learning," 2024, *arXiv:2405.02481*.
- [46] K. Hartikainen, X. Geng, T. Haarnoja, and S. Levine, "Dynamical distance learning for semi-supervised and unsupervised skill discovery," in *Proc. 8th Int. Conf. Learn. Representations*, 2020, pp. 7057–7073.

- [47] Q. Lin, R. Ma, and Y. Xu, "Complexity of an inexact proximal-point penalty method for constrained smooth non-convex optimization," *Comput. Optim. Appl.*, vol. 82, no. 1, pp. 175–224, 2022.
- [48] D. P. Bertsekas, "Nonlinear programming," *J. Oper. Res. Soc.*, vol. 48, no. 3, pp. 334–334, 1997.
- [49] J. Mei, C. Xiao, C. Szepesvari, and D. Schuurmans, "On the global convergence rates of softmax policy gradient methods," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6820–6829.
- [50] M. Plappert et al., "Multi-goal reinforcement learning: Challenging robotics environments and request for research," 2018, *arXiv:1802.09464*.
- [51] Y. Luo et al., "Relay hindsight experience replay: Self-guided continual reinforcement learning for sequential object manipulation tasks with sparse rewards," *Neurocomputing*, vol. 557, 2023, Art. no. 126620.
- [52] D. Ghosh et al., "Learning to reach goals via iterated supervised learning," in *Proc. 9th Int. Conf. Learn. Representations*, 2021, pp. 1–21.



Shaofu Yang (Member, IEEE) received the B.S. and M.S. degrees in applied mathematics from the Department of Mathematics, Southeast University, Nanjing, China, in 2010 and 2013, respectively, and the Ph.D. degree in engineering from the Department of Mechanical and Automation Engineering, The Chinese University of Hong Kong, Hong Kong, in 2016. In 2016, he was a Postdoctoral Fellow with the City University of Hong Kong, Hong Kong. He is currently an Associate Professor with the School of Computer Science and Engineering, Southeast University, China.

His research interests include distributed optimization, multi-agent learning, game theory, and their applications.



Xinglin Zhou received the B.S. degree in computer science and technology from the School of Computer Science and Technology, Dalian University of Technology, Dalian, China, in 2021, the Master of Science degree in artificial intelligence from the SEU-Monash Joint Graduate School, Southeast University, Suzhou, in 2024. He is currently an Algorithm Engineer with Zhongxing Telecommunications Equipment Company. His research interests include reinforcement learning from human feedback, large language models, and AI infrastructure development.



Jianye Hao (Senior Member, IEEE) is currently an Associate Professor with Tianjin University, Tianjin, China, and the Director of the Noah's Ark Decision making and Reasoning Laboratory, Huawei, Beijing China. The research of his team has been successfully applied in domains such as game artificial intelligence, E-commerce recommendation, network optimization, and supply chain optimization. His research interests include reinforcement learning and multi-agent systems. Prof. Hao was the recipient of the number of Best Paper awards such as ASE2019 and CoRL2020.



Yifu Yuan received the B.S. degree in computer science and technology from the School of Computer Science and Technology, Dalian University of Technology, Dalian, China, in 2021. He is currently working toward the second-year Ph.D. degree with the School of Intelligence and Computing, Tianjin University, Tianjin, China, under the supervision of Professor Jianye Hao. His research interests reinforcement learning and diffusion models.